



SWEN 221

A surreal, dreamlike landscape. In the foreground, a person stands on a dark, rocky outcrop, looking out over a vast, colorful sky. The sky is filled with large, billowing clouds in shades of pink, purple, and blue. A bright, glowing horizon line suggests a sunrise or sunset. In the upper right, a comet streaks across the sky, leaving a trail of light. The overall atmosphere is ethereal and otherworldly.

Marco Servetto
CO258

www.youtube.com/MarcoServetto



The wild world of Lambdas

Lambdas

Convenient syntax for anonymous nested classes

A lambda is an instance of an
anonymous nested class

A lambda is just an object!

Old and new

Comparators using long syntax for anonimus classes

```
Collections.sort(ls,new Comparator<String>(){  
    public int compare(String s1, String s2) {  
        return s1.compareToIgnoreCase(s2);  
    }});
```

- Convenient syntax for anonymous nested classes

Old and new

Comparators using long syntax for anonimus classes

```
Collections.sort(ls, new Comparator<String>(){  
    public int compare(String s1, String s2) {  
        return s1.compareToIgnoreCase(s2);  
    }});
```

Comparators using short syntax for anonimus classes

```
Collections.sort(ls, (s1, s2) -> s1.compareToIgnoreCase(s2));
```

- Convenient syntax for anonymous nested classes

Extensive use for event handler

```
SwingUtilities.invokeLater(new Runnable() {  
    public void run() {  
        MiniGui g= new MiniGui();  
        ...  
        JButton b= new JButton("-----Bar-----");  
        b.addActionListener(new ActionListener() {  
            public void actionPerformed(ActionEvent e) {  
                System.out.println("Button pressed");  
            }  
        });  
        ...});  
    }  
});
```

Extensive use for event handler

```
SwingUtilities.invokeLater(new Runnable() {  
    public void run() {  
        MiniGui g= new MiniGui();  
        ...  
        JButton b= new JButton("-----Bar-----");  
        b.addActionListener(new ActionListener() {  
            public void actionPerformed(ActionEvent e) {  
                System.out.println("Button pressed");  
            }  
        });  
        ...});  
    }  
});
```

Before and after

```
SwingUtilities.invokeLater(()->{  
    MiniGui g= new MiniGui();  
    ...  
    JButton b= new JButton("-----Bar-----");  
    b.addActionListener(e->System.out.println("Button pressed"));  
    ...});
```


Alternatives for syntax

```
p-> p.getAge()
```

Parameter name, arrow, expression

```
(p1,p2)-> p1.getAge() > p2.getAge()
```

```
()-> System.currentTimeMillis()
```

```
(customer,product)-> {  
    if (customer.getAge() < 25 && product.hasAlcohol()){  
        return "Please, show me your id!"  
    }  
    return "Do you need a receipt?"  
}
```

```
p-> p.getAge()
```

```
==
```

```
(Person p)-> p.getAge()
```

```
==
```

```
p->{ return p.getAge(); }
```



Alternatives for syntax

```
p-> p.getAge()
```

```
(p1,p2)-> p1.getAge() > p2.getAge()
```

```
()-> System.currentTimeMillis()
```

```
(customer,product)-> {  
  if (customer.getAge() < 25 &&  
    return "Please, show me your  
  }  
  return "Do you need a receipt?"  
}
```

```
p-> p.getAge()
```

```
==
```

```
(Person p)-> p.getAge()
```

```
==
```

```
p->{ return p.getAge(); }
```

Two or more parameters
require parenthesis



Alternatives for syntax

```
p-> p.getAge()
```

```
(p1,p2)-> p1.getAge() > p2.getAge()
```

```
()-> System.currentTimeMillis()
```

```
(customer,product)-> {  
  if (customer.getAge() < 25 && product.hasAlcohol()){  
    return "Please, show me your id!"  
  }  
  return "Do you need a receipt?"  
}
```

```
p-> p.getAge()
```

```
==
```

```
(Person p)-> p.getAge()
```

```
==
```

```
p->{ return p.getAge(); }
```

Zero arguments:
`open, close, arrow`
Very common



Alternatives for syntax

Block of
statements
instead of
expression

```
p-> p.getAge()
```

```
(p1,p2)-> p1.getAge() > p2.getAge()
```

```
()-> System.currentTimeMillis()
```

```
(customer,product)-> {  
    if (customer.getAge() < 25 && product.hasAlcohol()){  
        return "Please, show me your id!"  
    }  
    return "Do you need a receipt?"  
}
```

```
p-> p.getAge()
```

```
==
```

```
(Person p)-> p.getAge()
```

```
==
```

```
p->{ return p.getAge(); }
```



Alternatives for syntax

Three
equivalent
forms:

```
p-> p.getAge()
```

```
(p1,p2)-> p1.getAge() > p2.getAge()
```

```
()-> System.currentTimeMillis()
```

```
(customer,product)-> {  
    if (customer.getAge() < 25 && product.hasAlcohol()){  
        return "Please, show me your id!"  
    }  
    return "Do you need a receipt?"  
}
```

```
p-> p.getAge()
```

```
==
```

```
(Person p)-> p.getAge()
```

```
==
```

```
p->{ return p.getAge(); }
```

Basic

Explicit parameter type

Block and return



More alternative syntax :-)

```
String::length           // instance method
System::currentTimeMillis // static method
List<String>::size        // explicit type arguments for generic type
List::size                // inferred type arguments for generic type
int[]::clone
T::fieldName
System.out::println
"abc"::length
foo[x]::bar
(test ? list.replaceAll(String::trim) : list) :: iterator
super::toString
String::valueOf           // overload resolution needed
Arrays::sort              // type arguments inferred from context
Arrays::<String>sort      // explicit type arguments
ArrayList<String>::new     // constructor for parameterized type
ArrayList::new            // inferred type arguments for generic class
Foo::<Integer>new         // explicit type arguments for generic constructor
Bar<String>::<Integer>new  // generic class, generic constructor
Outer.Inner::new         // inner class constructor
int[]::new                // array creation
```

Syntax and types

- Can use short syntax to implement any ***Functional Interface:***
 - An interface that needs exactly one method implementation to be fully satisfied.

Syntax and types

- Can use short syntax to implement any ***Functional Interface***:
 - An interface that needs exactly one method implementation to be fully satisfied.
- Examples (Java before 8):
Comparable<T>, Comparator<T>,
Runnable, Callable<V>, AutoCloseable

Syntax and types

- Can use short syntax to implement any ***Functional Interface***:
 - An interface that needs exactly one method implementation to be fully satisfied.
- Examples (Java before 8):
Comparable<T>, Comparator<T>,
Runnable, Callable<V>, AutoCloseable
- In Java8, > 40 different functional interfaces:
 - We will need to learn the most important ones!

<https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/function/package-summary.html>

Functional interfaces

Main Java 8 functional interface:

- `Function<T, R>`
- A function from
 - type T (parameter)
 - to type R (return type)
- Some composition behavior provided!

Code of Function<T,R> in `java.util.function`

```
interface Function<T, R> {  
  
    R apply(T t); //method still to define, often using the new lambda syntax  
  
    static <T> Function<T, T> identity(){ return t -> t; }  
  
    default <V> Function<V, R> compose( Function<? super V, ? extends T> before ){  
        return (V v) -> this.apply(before.apply(v));  
    }  
  
    default <V> Function<T, V> andThen( Function<? super R, ? extends V> after ){  
        return (T t) -> after.apply(this.apply(t));  
    }  
}
```

- Minimal code, but not “simple”



なに!!

Code of Function<T,R> in `java.util.function`

```
interface Function<T, R> {  
  
    R apply(T t); //method still to define, often using the new lambda syntax  
  
    static <T> Function<T, T> identity(){ return t -> t; }  
  
    default <V> Function<V, R> compose( Function<? super V, ? extends T> before ){  
        return (V v) -> this.apply(before.apply(v));  
    }  
  
    default <V> Function<T, V> andThen( Function<? super R, ? extends V> after ){  
        return (T t) -> after.apply(this.apply(t));  
    }  
}
```

- Yes, this is an actual extract from the Java standard library.
- This is how the Java authors are telling us to code.

Simplified code of Function<T,R>

```
interface Function<T, R> {  
    R apply(T t); //method still to define, often using the new lambda syntax  
  
    static <T> Function<T, T> identity(){ return t -> t; }  
  
    default <V> Function<V, R> compose( Function<V, T> before ){  
        return v -> this.apply(before.apply(v));  
    }  
  
    default <V> Function<T, V> andThen( Function<R, V> after ){  
        return t -> after.apply(this.apply(t));  
    }  
}
```

- Without the question marks becomes more understandable

Code of Function<T,R> in `java.util.function`

```
Function<Integer,Integer> multiply2= x-> x * 2;
```

```
Function<Integer,Integer> add2= x-> x + 2;
```

```
System.out.println(  
    multiply2.andThen(add2).apply(1)); // (1 * 2) + 2 = 4
```

```
System.out.println(  
    add2.andThen(multiply2).apply(1)); // (1 + 2) * 2 = 6
```

```
System.out.println(  
    multiply2.andThen(multiply2).apply(1)); // 1 * 2 * 2 = 4
```

```
System.out.println(  
    add2.compose(multiply2).apply(1)); // (1 * 2) + 2 = 4  
//==multiply2.andThen(add2)
```

- Simpler when sub/super types are not involved

More functional interfaces

We have seen:

`Function`

Now:

`Consumer<T>`

- A kind of function that eats up a value.
- Has `accept` method returning void
- Has an `andThen` method to compose `Consumers`:
values accepted by a composed consumer are accepted by both consumers

More functional interfaces

We have seen:

`Function, Consumer`

Now:

`Supplier<T>`

- A kind of function that takes no arguments.
- Has a `get` method returning a value of type `T`
- Very useful, similar to factory pattern. Used very often!
- A supplier looks like this: `() -> new Point(12,3)`

More functional interfaces

We have seen:

`Function`, `Consumer`, `Supplier`

Now:

`Predicate<T>`

- A kind of function that takes 1 argument
- Has a `test` method returning a boolean
- Has `and`, `or`, `negate` methods allowing to compose `Predicates`.

Guided exercise

```
public static int reduceSum(List<Integer> list){  
    assert !list.isEmpty(); //or if(list.isEmpty()){ throw... }  
    int res= list.get(0);  
    for(int i= 1; i < list.size(); i++){ res= res + list.get(i); }  
    return res;  
}
```

Guided exercise

- In this code there is a lot of repetition!
- Use lambdas and factorize the code!

```
public static int reduceSum(List<Integer> list){
    assert !list.isEmpty(); //or if(list.isEmpty()){ throw... }
    int res= list.get(0);
    for(int i= 1; i < list.size(); i++){ res= res + list.get(i); }
    return res;
}
public static int reduceMul(List<Integer> list){
    assert !list.isEmpty(); //or if(list.isEmpty()){ throw... }
    int res= list.get(0);
    for(int i= 1; i < list.size(); i++){ res= res * list.get(i); }
    return res;
}
```

Guided exercise

- In this code there is a lot of repetition!
- Use lambdas and factorize the code!

```
public static int reduceSum(List<Integer> list){
    assert !list.isEmpty(); //or if(list.isEmpty()){ throw... }
    int res= list.get(0);
    for(int i= 1; i < list.size(); i++){ res= res + list.get(i); }
    return res;
}

public static int reduceMul(List<Integer> list){
    assert !list.isEmpty(); //or if(list.isEmpty()){ throw... }
    int res= list.get(0);
    for(int i= 1; i < list.size(); i++){ res= res * list.get(i); }
    return res;
}
```

This is the only
difference in
sum/mul



Guided exercise

- Can we write `Reduce.of(list, lambda)`?

```
Reduce.of( list, (a,b)-> a + b );
```

Guided exercise

- Can we write `Reduce.of(list, lambda)`?

```
public static void main(String[] arg){  
    List<Integer> list= List.of(1,2,3,4,5,6,7,3);  
  
    System.out.println( Reduce.of(list,(a,b)-> a + b) );  
  
    System.out.println( Reduce.of(list,(a,b)-> a * b) );  
  
    System.out.println( Reduce.of(list,(a,b)->  
        {if (a > b){ return a; } return b; }));  
}
```

Guided exercise

- Can we write `Reduce.of(list, lambda)`?

```
public interface Reduce<T> {  
  
    T apply(T e1, T e2);  
  
    public static <T> T of(List<T> list, Reduce<T> fun){  
        assert !list.isEmpty(); //or if(..){ throw... }  
        T res= list.get(0);  
        for (int i= 1; i < list.size(); i++){ res = fun.apply(res, list.get(i)); }  
        return res;  
    }  
}  
  
//compare it with the specific code of before:  
//assert !list.isEmpty();  
//int res= list.get(0);  
//for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }  
//return res;
```


A close-up, stylized illustration of Vegeta's face. He is wearing dark sunglasses with gold-colored frames. His hair is spiky and black. The background is a vibrant sunset or sunrise with orange and yellow clouds. The text "Visualizing code" is overlaid in the bottom left corner in a light blue, sans-serif font with a black outline.

Visualizing code

Visualizing code: the empty list case

```
public static int reduceSum(List<Integer> list){  
    assert !list.isEmpty();  
    int res= list.get(0);  
    for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }  
    return res;  
}  
  
public static void main(String[] arg){  
    List<Integer> list= List.of();  
    System.out.println( reduceSum(list) );  
}
```

Visualizing code: the empty list case

```
public static int reduceSum(List<Integer> list){
    assert !list.isEmpty();
    int res= list.get(0);
    for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }
    return res;
}

public static void main(String[] arg){
    //List<Integer> list = List.of(); we can forget about this since it is not used elsewhere
    System.out.println( {
        List<Integer> list= List.of();
        assert !list.isEmpty();
        int res= list.get(0);
        for(int i= 1; i < list.size(); i++){ res = res + list.get(i); }
        return res;
    });
}
```

Visualizing code: the empty list case

```
public static int reduceSum(List<Integer> list){
    assert !list.isEmpty();
    int res= list.get(0);
    for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }
    return res;
}

public static void main(String[] arg){

    System.out.println( {
        List<Integer> list= List.of();
        assert !true;
        int res= list.get(0);
        for(int i= 1; i < list.size(); i++){ res = res + list.get(i); }
        return res;
    });
}
```

Visualizing code: the empty list case

```
public static int reduceSum(List<Integer> list){
    assert !list.isEmpty();
    int res= list.get(0);
    for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }
    return res;
}

public static void main(String[] arg){

    System.out.println( {
        List<Integer> list= List.of();
        assert false;
        int res= list.get(0);
        for(int i= 1; i < list.size(); i++){ res = res + list.get(i); }
        return res;
    });
}
```

Visualizing code: the empty list case

```
public static int reduceSum(List<Integer> list){
    assert !list.isEmpty();
    int res= list.get(0);
    for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }
    return res;
}

public static void main(String[] arg){

    System.out.println( {
        List<Integer> list= List.of();
        throw new AssertionError();
        int res= list.get(0);
        for(int i= 1; i < list.size(); i++){ res = res + list.get(i); }
        return res;
    });
}
```

Visualizing code: the empty list case

```
public static int reduceSum(List<Integer> list){
    assert !list.isEmpty();
    int res= list.get(0);
    for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }
    return res;
}

public static void main(String[] arg){

    System.out.println( {

        throw new AssertionError();

    });
}
```


Visualizing code: the empty list case

```
public static int reduceSum(List<Integer> list){  
    assert !list.isEmpty();  
    int res= list.get(0);  
    for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }  
    return res;  
}
```

```
public static void main(String[] arg){  
  
    throw new AssertionError();  
  
}
```

//ENDS with AssertionError in line 2

Visualizing code: the singleton list case

```
public static int reduceSum(List<Integer> list){
    assert !list.isEmpty();
    int res= list.get(0);
    for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }
    return res;
}

public static void main(String[] arg){
    List<Integer> list= List.of(1);
    System.out.println( reduceSum(list) );
}
```

Visualizing code: the singleton list case

```
public static int reduceSum(List<Integer> list){
    assert !list.isEmpty();
    int res= list.get(0);
    for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }
    return res;
}

public static void main(String[] arg){
    System.out.println( {
        List<Integer> list= List.of(1);
        assert !list.isEmpty();
        int res= list.get(0);
        for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }
        return res;
    });
}
```

Visualizing code: the singleton list case

```
public static int reduceSum(List<Integer> list){
    assert !list.isEmpty();
    int res= list.get(0);
    for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }
    return res;
}

public static void main(String[] arg){
    System.out.println( {
        List<Integer> list= List.of(1);
        assert !false;
        int res= list.get(0);
        for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }
        return res;
    });
}
```

Visualizing code: the singleton list case

```
public static int reduceSum(List<Integer> list){
    assert !list.isEmpty();
    int res= list.get(0);
    for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }
    return res;
}

public static void main(String[] arg){
    System.out.println( {
        List<Integer> list= List.of(1);
        assert true;
        int res= list.get(0);
        for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }
        return res;
    });
}
```

Visualizing code: the singleton list case

```
public static int reduceSum(List<Integer> list){
    assert !list.isEmpty();
    int res= list.get(0);
    for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }
    return res;
}

public static void main(String[] arg){
    System.out.println( {
        List<Integer> list= List.of(1);
        int res= list.get(0);
        for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }
        return res;
    });
}
```

Visualizing code: the singleton list case

```
public static int reduceSum(List<Integer> list){
    assert !list.isEmpty();
    int res= list.get(0);
    for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }
    return res;
}

public static void main(String[] arg){
    System.out.println( {
        List<Integer> list= List.of(1);
        int res= 1;
        for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }
        return res;
    });
}
```


Visualizing code: the singleton list case

```
public static int reduceSum(List<Integer> list){
    assert !list.isEmpty();
    int res= list.get(0);
    for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }
    return res;
}

public static void main(String[] arg){
    System.out.println( {
        List<Integer> list= List.of(1);
        int res= 1;
        if (1 < list.size()){
            res = res + list.get(1);
            i++;
            for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }
        }
        return res;
    });
}
```

Visualizing code: the singleton list case

```
public static int reduceSum(List<Integer> list){
    assert !list.isEmpty();
    int res= list.get(0);
    for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }
    return res;
}

public static void main(String[] arg){
    System.out.println( {
        List<Integer> list= List.of(1);
        int res= 1;
        if (1 < 1){
            res = res + list.get(1);
            i++;
            for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }
        }
        return res;
    });
}
```

Visualizing code: the singleton list case

```
public static int reduceSum(List<Integer> list){
    assert !list.isEmpty();
    int res= list.get(0);
    for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }
    return res;
}

public static void main(String[] arg){
    System.out.println( {
        List<Integer> list= List.of(1);
        int res= 1;
        if (false){
            res = res + list.get(1);
            i++;
            for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }
        }
        return res;
    });
}
```

Visualizing code: the singleton list case

```
public static int reduceSum(List<Integer> list){
    assert !list.isEmpty();
    int res= list.get(0);
    for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }
    return res;
}

public static void main(String[] arg){
    System.out.println( {
        List<Integer> list= List.of(1);
        int res= 1;
        return res;
    });
}
```

Visualizing code: the singleton list case

```
public static int reduceSum(List<Integer> list){
    assert !list.isEmpty();
    int res= list.get(0);
    for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }
    return res;
}

public static void main(String[] arg){
    System.out.println( {

        int res= 1;
        return 1;
    });
}
```

Visualizing code: the singleton list case

```
public static int reduceSum(List<Integer> list){  
    assert !list.isEmpty();  
    int res= list.get(0);  
    for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }  
    return res;  
}  
  
public static void main(String[] arg){  
    System.out.println(1);  
}
```

Longer list, skipping some steps

```
public static int reduceSum(List<Integer> list){
    assert !list.isEmpty();
    int res= list.get(0);
    for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }
    return res;
}

public static void main(String[] arg){
    List<Integer> list= List.of(1,2,3,4,5,6,7,3);

    System.out.println( reduceSum(list) );
}
```

Longer list, skipping some steps

```
public static int reduceSum(List<Integer> list){
    assert !list.isEmpty();
    int res= list.get(0);
    for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }
    return res;
}

public static void main(String[] arg){

    System.out.println({
        List<Integer> list= List.of(1,2,3,4,5,6,7,3);
        assert !list.isEmpty();
        int res= list.get(0);
        for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }
        return res;
    });
}
```


Longer list, skipping some steps

```
public static int reduceSum(List<Integer> list){
    assert !list.isEmpty();
    int res= list.get(0);
    for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }
    return res;
}

public static void main(String[] arg){

    System.out.println({
        List<Integer> list= List.of(1,2,3,4,5,6,7,3);
        int res= list.get(0);
        for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }
        return res;
    });
}
```

Longer list, skipping some steps

```
public static int reduceSum(List<Integer> list){
    assert !list.isEmpty();
    int res= list.get(0);
    for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }
    return res;
}

public static void main(String[] arg){

    System.out.println({
        List<Integer> list= List.of(1,2,3,4,5,6,7,3);
        int res= 1;
        for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }
        return res;
    });
}
```

Longer list, skipping some steps

```
public static int reduceSum(List<Integer> list){
    assert !list.isEmpty();
    int res= list.get(0);
    for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }
    return res;
}

public static void main(String[] arg){

    System.out.println({
        List<Integer> list= List.of(1,2,3,4,5,6,7,3);
        int res= 1;
        if (1 < list.size()){
            res = res + list.get(1);
            i++;
            for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }
        }
        return res;
    });
}
```

Longer list, skipping some steps

```
public static int reduceSum(List<Integer> list){
    assert !list.isEmpty();
    int res= list.get(0);
    for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }
    return res;
}

public static void main(String[] arg){

    System.out.println({
        List<Integer> list= List.of(1,2,3,4,5,6,7,3);
        int res= 1;
        if (1 < 8){
            res = res + list.get(1);
            i++;
            for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }
        }
        return res;
    });
}
```

Longer list, skipping some steps

```
public static int reduceSum(List<Integer> list){
    assert !list.isEmpty();
    int res= list.get(0);
    for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }
    return res;
}

public static void main(String[] arg){

    System.out.println({
        List<Integer> list= List.of(1,2,3,4,5,6,7,3);
        int res= 1;
        if (true){
            res = res + list.get(1);
            i++;
            for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }
        }
        return res;
    });
}
```

Longer list, skipping some steps

```
public static int reduceSum(List<Integer> list){
    assert !list.isEmpty();
    int res= list.get(0);
    for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }
    return res;
}

public static void main(String[] arg){

    System.out.println({
        List<Integer> list= List.of(1,2,3,4,5,6,7,3);
        int res= 1;

        res = res + list.get(1);
        i++;
        for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }

        return res;
    });
}
```

Longer list, skipping some steps

```
public static int reduceSum(List<Integer> list){
    assert !list.isEmpty();
    int res= list.get(0);
    for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }
    return res;
}

public static void main(String[] arg){

    System.out.println({
        List<Integer> list= List.of(1,2,3,4,5,6,7,3);
        int res= 1;

        res = 1 + 2;
        i++;
        for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }

        return res;
    });
}
```

Longer list, skipping some steps

```
public static int reduceSum(List<Integer> list){
    assert !list.isEmpty();
    int res= list.get(0);
    for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }
    return res;
}

public static void main(String[] arg){

    System.out.println({
        List<Integer> list= List.of(1,2,3,4,5,6,7,3);
        int res= 3;

        i++;
        for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }

        return res;
    });
}
```


Longer list, skipping some steps

```
public static int reduceSum(List<Integer> list){
    assert !list.isEmpty();
    int res= list.get(0);
    for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }
    return res;
}

public static void main(String[] arg){

    System.out.println({
        List<Integer> list= List.of(1,2,3,4,5,6,7,3);
        int res= 3;

        for (int i= 2; i < list.size(); i++){ res = res + list.get(i); }

        return res;
    });
}
```

Longer list, skipping some steps

```
public static int reduceSum(List<Integer> list){
    assert !list.isEmpty();
    int res= list.get(0);
    for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }
    return res;
}

public static void main(String[] arg){

    System.out.println({
        List<Integer> list= List.of(1,2,3,4,5,6,7,3);
        int res= 3;
        res = res + list.get(2); //skipping more steps
        i++;
        for (int i= 2; i < list.size(); i++){ res = res + list.get(i); }

        return res;
    });
}
```

Longer list, skipping some steps

```
public static int reduceSum(List<Integer> list){
    assert !list.isEmpty();
    int res= list.get(0);
    for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }
    return res;
}

public static void main(String[] arg){

    System.out.println({
        List<Integer> list= List.of(1,2,3,4,5,6,7,3);
        int res= 3;
        res = 3 + 3;
        i++;
        for (int i= 2; i < list.size(); i++){ res = res + list.get(i); }

        return res;
    });
}
```

Longer list, skipping some steps

```
public static int reduceSum(List<Integer> list){
    assert !list.isEmpty();
    int res= list.get(0);
    for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }
    return res;
}

public static void main(String[] arg){

    System.out.println({
        List<Integer> list= List.of(1,2,3,4,5,6,7,3);
        int res= 6;

        for (int i= 3; i < list.size(); i++){ res = res + list.get(i); }

        return res;
    });
}
```

Longer list, skipping some steps

```
public static int reduceSum(List<Integer> list){
    assert !list.isEmpty();
    int res= list.get(0);
    for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }
    return res;
}

public static void main(String[] arg){

    System.out.println({
        List<Integer> list= List.of(1,2,3,4,5,6,7,3);
        int res= 6;
        res = res + list.get(3);    //4
        res = res + list.get(4);    //5
        res = res + list.get(5);    //6
        res = res + list.get(6);    //7
        res = res + list.get(7);    //3
        //for (int i= 3; i < list.size(); i++){ res = res + list.get(i); }//Unfolding

        return res;
    });
}
```

Longer list, skipping some steps

```
public static int reduceSum(List<Integer> list){
    assert !list.isEmpty();
    int res= list.get(0);
    for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }
    return res;
}

public static void main(String[] arg){

    System.out.println({
        List<Integer> list= List.of(1,2,3,4,5,6,7,3);
        int res= 6;
        res = 6 + 4;
        res = res + 5;
        res = res + 6;
        res = res + 7;
        res = res + 3;
        //for (int i= 3; i < list.size(); i++){ res = res + list.get(i); }//Unfolding

        return res;
    });
}
```

Longer list, skipping some steps

```
public static int reduceSum(List<Integer> list){
    assert !list.isEmpty();
    int res= list.get(0);
    for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }
    return res;
}

public static void main(String[] arg){

    System.out.println({
        List<Integer> list= List.of(1,2,3,4,5,6,7,3);
        int res= 10;
        res = res + 5;
        res = res + 6;
        res = res + 7;
        res = res + 3;
        //for (int i= 3; i < list.size(); i++){ res = res + list.get(i); }//Unfolding

        return res;
    });
}
```

Longer list, skipping some steps

```
public static int reduceSum(List<Integer> list){
    assert !list.isEmpty();
    int res= list.get(0);
    for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }
    return res;
}

public static void main(String[] arg){

    System.out.println({
        List<Integer> list= List.of(1,2,3,4,5,6,7,3);
        int res= 10;
        res = 10 + 5;
        res = res + 6;
        res = res + 7;
        res = res + 3;
        //for (int i= 3; i < list.size(); i++){ res = res + list.get(i); }//Unfolding

        return res;
    });
}
```


Longer list, skipping some steps

```
public static int reduceSum(List<Integer> list){
    assert !list.isEmpty();
    int res= list.get(0);
    for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }
    return res;
}

public static void main(String[] arg){

    System.out.println({
        List<Integer> list= List.of(1,2,3,4,5,6,7,3);
        int res= 15;
        res = 15 + 6;
        res = res + 7;
        res = res + 3;
        //for (int i= 3; i < list.size(); i++){ res = res + list.get(i); }//Unfolding

        return res;
    });
}
```

Longer list, skipping some steps

```
public static int reduceSum(List<Integer> list){
    assert !list.isEmpty();
    int res= list.get(0);
    for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }
    return res;
}

public static void main(String[] arg){

    System.out.println({
        List<Integer> list= List.of(1,2,3,4,5,6,7,3);
        int res= 21;
        res = 21 + 7;
        res = res + 3;
        //for (int i= 3; i < list.size(); i++){ res = res + list.get(i); }//Unfolding

        return res;
    });
}
```

Longer list, skipping some steps

```
public static int reduceSum(List<Integer> list){
    assert !list.isEmpty();
    int res= list.get(0);
    for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }
    return res;
}

public static void main(String[] arg){

    System.out.println({
        List<Integer> list= List.of(1,2,3,4,5,6,7,3);
        int res= 28;
        res = 28 + 3;
        //for (int i= 3; i < list.size(); i++){ res = res + list.get(i); }//Unfolding

        return res;
    });
}
```

Longer list, skipping some steps

```
public static int reduceSum(List<Integer> list){
    assert !list.isEmpty();
    int res= list.get(0);
    for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }
    return res;
}

public static void main(String[] arg){

    System.out.println({
        List<Integer> list= List.of(1,2,3,4,5,6,7,3);
        int res= 31;
        //for (int i= 3; i < list.size(); i++){ res = res + list.get(i); }//Unfolding
        return res;
    });
}
```

Longer list, skipping some steps

```
public static int reduceSum(List<Integer> list){
    assert !list.isEmpty();
    int res= list.get(0);
    for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }
    return res;
}

public static void main(String[] arg){

    System.out.println({
        int res= 31;
        return 31;
    });
}
```

Longer list, skipping some steps

```
public static int reduceSum(List<Integer> list){
    assert !list.isEmpty();
    int res= list.get(0);
    for (int i= 1; i < list.size(); i++){ res = res + list.get(i); }
    return res;
}

public static void main(String[] arg){

    System.out.println(31);

}
```

Guided exercise, Max variant

```
public static int reduceMax(List<Integer> list){
    assert !list.isEmpty();
    int res= list.get(0);
    for (int i= 1; i < list.size(); i++){ if (res < list.get(i)){ res = list.get(i); } }
    //equivalent to          res = res < list.get(i) ? list.get(i) : max;
    return res;
}

public static void main(String[] arg){
    List<Integer> list= List.of(1,2,3,4,5,6,7,3);

    System.out.println( reduceMax(list) );
}
```

Guided exercise, Max variant

```
public static int reduceMax(List<Integer> list){
    assert !list.isEmpty();
    int res= list.get(0);
    for (int i= 1; i < list.size(); i++){ if (res < list.get(i)){ res = list.get(i); } }
    //equivalent to          res = res < list.get(i) ? list.get(i) : max;
    return res;
}

public static void main(String[] arg){
    List<Integer> list= List.of(1,2,3,4,5,6,7,3);

    System.out.println( {
        int res= list.get(0);
        for (int i= 1; i < list.size(); i++){ if (res < list.get(i)){ res = list.get(i);} }
        return res;
    });
}
```


Guided exercise, Max variant

```
public static int reduceMax(List<Integer> list){
    assert !list.isEmpty();
    int res= list.get(0);
    for (int i= 1; i < list.size(); i++){ if (res < list.get(i)){ res = list.get(i); } }
    //equivalent to          res = res < list.get(i) ? list.get(i) : max;
    return res;
}

public static void main(String[] arg){
    System.out.println( {
        List<Integer> list= List.of(1,2,3,4,5,6,7,3);
        int res= 1;
        if (res < list.get(1)){ res = list.get(1); }
        if (res < list.get(2)){ res = list.get(2); }
        if (res < list.get(3)){ res = list.get(3); }
        if (res < list.get(4)){ res = list.get(4); }
        if (res < list.get(5)){ res = list.get(5); }
        if (res < list.get(6)){ res = list.get(6); }
        if (res < list.get(7)){ res = list.get(7); }
        return res;
    });
}
```

Guided exercise, Max variant

```
public static int reduceMax(List<Integer> list){
    assert !list.isEmpty();
    int res= list.get(0);
    for (int i= 1; i < list.size(); i++){ if (res < list.get(i)){ res = list.get(i); } }
    //equivalent to          res = res < list.get(i) ? list.get(i) : max;
    return res;
}

public static void main(String[] arg){
    System.out.println( {
        List<Integer> list= List.of(1,2,3,4,5,6,7,3);
        int res= 1;
        if (res < 2){ res = 2; }
        if (res < 3){ res = 3; }
        if (res < 4){ res = 4; }
        if (res < 5){ res = 5; }
        if (res < 6){ res = 6; }
        if (res < 7){ res = 7; }
        if (res < 3){ res = 3; }
        return res;
    });
}
```

Guided exercise, Max variant

```
public static int reduceMax(List<Integer> list){
    assert !list.isEmpty();
    int res= list.get(0);
    for (int i= 1; i < list.size(); i++){ if (res < list.get(i)){ res = list.get(i); } }
    //equivalent to          res = res < list.get(i) ? list.get(i) : max;
    return res;
}

public static void main(String[] arg){
    System.out.println( {
        List<Integer> list= List.of(1,2,3,4,5,6,7,3);
        int res= 1;
        if (1 < 2){ res = 2; }
        if (res < 3){ res = 3; }
        if (res < 4){ res = 4; }
        if (res < 5){ res = 5; }
        if (res < 6){ res = 6; }
        if (res < 7){ res = 7; }
        if (res < 3){ res = 3; }
        return res;
    });
}
```

Guided exercise, Max variant

```
public static int reduceMax(List<Integer> list){
    assert !list.isEmpty();
    int res= list.get(0);
    for (int i= 1; i < list.size(); i++){ if (res < list.get(i)){ res = list.get(i); } }
    //equivalent to          res = res < list.get(i) ? list.get(i) : max;
    return res;
}

public static void main(String[] arg){
    System.out.println( {
        List<Integer> list= List.of(1,2,3,4,5,6,7,3);
        int res= 2;

        if (2 < 3){ res = 3; }
        if (res < 4){ res = 4; }
        if (res < 5){ res = 5; }
        if (res < 6){ res = 6; }
        if (res < 7){ res = 7; }
        if (res < 3){ res = 3; }
        return res;
    });
}
```

Guided exercise, Max variant

```
public static int reduceMax(List<Integer> list){
    assert !list.isEmpty();
    int res= list.get(0);
    for (int i= 1; i < list.size(); i++){ if (res < list.get(i)){ res = list.get(i); } }
    //equivalent to          res = res < list.get(i) ? list.get(i) : max;
    return res;
}

public static void main(String[] arg){
    System.out.println( {
        List<Integer> list= List.of(1,2,3,4,5,6,7,3);
        int res= 3;

        if (3 < 4){ res = 4; }
        if (res < 5){ res = 5; }
        if (res < 6){ res = 6; }
        if (res < 7){ res = 7; }
        if (res < 3){ res = 3; }
        return res;
    });
}
```

Guided exercise, Max variant

```
public static int reduceMax(List<Integer> list){
    assert !list.isEmpty();
    int res= list.get(0);
    for (int i= 1; i < list.size(); i++){ if (res < list.get(i)){ res = list.get(i); } }
    //equivalent to          res = res < list.get(i) ? list.get(i) : max;
    return res;
}

public static void main(String[] arg){
    System.out.println( {
        List<Integer> list= List.of(1,2,3,4,5,6,7,3);
        int res= 4;

        if (4 < 5){ res = 5; }
        if (res < 6){ res = 6; }
        if (res < 7){ res = 7; }
        if (res < 3){ res = 3; }
        return res;
    });
}
```

Guided exercise, Max variant

```
public static int reduceMax(List<Integer> list){
    assert !list.isEmpty();
    int res= list.get(0);
    for (int i= 1; i < list.size(); i++){ if (res < list.get(i)){ res = list.get(i); } }
    //equivalent to          res = res < list.get(i) ? list.get(i) : max;
    return res;
}

public static void main(String[] arg){
    System.out.println( {
        List<Integer> list= List.of(1,2,3,4,5,6,7,3);
        int res= 5;

        if (5 < 6){ res = 6; }
        if (res < 7){ res = 7; }
        if (res < 3){ res = 3; }
        return res;
    });
}
```

Guided exercise, Max variant

```
public static int reduceMax(List<Integer> list){
    assert !list.isEmpty();
    int res= list.get(0);
    for (int i= 1; i < list.size(); i++){ if (res < list.get(i)){ res = list.get(i); } }
    //equivalent to          res = res < list.get(i) ? list.get(i) : max;
    return res;
}

public static void main(String[] arg){
    System.out.println( {
        List<Integer> list= List.of(1,2,3,4,5,6,7,3);
        int res= 6;

        if (6 < 7){ res = 7; }
        if (res < 3){ res = 3; }
        return res;
    });
}
```


Guided exercise, Max variant

```
public static int reduceMax(List<Integer> list){
    assert !list.isEmpty();
    int res= list.get(0);
    for (int i= 1; i < list.size(); i++){ if (res < list.get(i)){ res = list.get(i); } }
    //equivalent to          res = res < list.get(i) ? list.get(i) : max;
    return res;
}

public static void main(String[] arg){
    System.out.println( {
        List<Integer> list= List.of(1,2,3,4,5,6,7,3);
        int res= 7;

        if (7 < 3){ res = 3; }
        return res;
    });
}
```

Guided exercise, Max variant

```
public static int reduceMax(List<Integer> list){
    assert !list.isEmpty();
    int res= list.get(0);
    for (int i= 1; i < list.size(); i++){ if (res < list.get(i)){ res = list.get(i); } }
    //equivalent to          res = res < list.get(i) ? list.get(i) : max;
    return res;
}

public static void main(String[] arg){
    System.out.println( {
        List<Integer> list= List.of(1,2,3,4,5,6,7,3);
        int res= 7;

        if (false){ res = 3; }
        return res;
    });
}
```

Guided exercise, Max variant

```
public static int reduceMax(List<Integer> list){
    assert !list.isEmpty();
    int res= list.get(0);
    for (int i= 1; i < list.size(); i++){ if (res < list.get(i)){ res = list.get(i); } }
    //equivalent to          res = res < list.get(i) ? list.get(i) : max;
    return res;
}

public static void main(String[] arg){
    System.out.println( {
        List<Integer> list= List.of(1,2,3,4,5,6,7,3);
        int res= 7;

        return res;
    });
}
```

Guided exercise, Max variant

```
public static int reduceMax(List<Integer> list){
    assert !list.isEmpty();
    int res= list.get(0);
    for (int i= 1; i < list.size(); i++){ if (res < list.get(i)){ res = list.get(i); } }
    //equivalent to          res = res < list.get(i) ? list.get(i) : max;
    return res;
}

public static void main(String[] arg){
    System.out.println( 7 );
}
```

Guided exercise, Reduce.of

```
public interface Reduce<T> {  
  
    T apply(T e1, T e2);  
  
    public static <T> T of(List<T> list, Reduce<T> fun){  
        assert !list.isEmpty();  
        T res= list.get(0);  
        for (int i= 1; i < list.size(); i++){ res = fun.apply(res, list.get(i)); }  
        return res;  
    }  
}  
  
public static void main(String[] arg){  
    List<Integer> list= List.of(1,2,3,4,5,6,7,3);  
  
    System.out.println( Reduce.of(list,(a,b)-> a + b) );  
}
```

Guided exercise, Reduce.of

```
public interface Reduce<T> {  
  
    T apply(T e1, T e2);  
  
    public static <T> T of(List<T> list, Reduce<T> fun){  
        assert !list.isEmpty();  
        T res= list.get(0);  
        for (int i= 1; i < list.size(); i++){ res = fun.apply(res, list.get(i)); }  
        return res;  
    }  
}  
  
public static void main(String[] arg){  
  
    System.out.println( {  
        List<Integer> list= List.of(1,2,3,4,5,6,7,3);  
        Reduce<T> fun= (a,b)-> a + b;  
        assert !list.isEmpty();  
        T res= list.get(0);  
        for (int i= 1;i < list.size(); i++){ res = fun.apply(res, list.get(i)); }  
        return res;  
    });  
}
```

Guided exercise, Reduce.of

```
public interface Reduce<T> {  
  
    T apply(T e1, T e2);  
  
    public static <T> T of(List<T> list, Reduce<T> fun){  
        assert !list.isEmpty();  
        T res= list.get(0);  
        for (int i= 1; i < list.size(); i++){ res = fun.apply(res, list.get(i)); }  
        return res;  
    }  
}  
  
public static void main(String[] arg){  
  
    System.out.println( {  
        List<Integer> list= List.of(1,2,3,4,5,6,7,3);  
        Reduce<T> fun= (a,b)-> a + b;  
        T res= 1;  
        for (int i= 1; i < 8; i++){ res = fun.apply(res, list.get(i)); }  
        return res;  
    });  
}
```

Guided exercise, Reduce.of

```
public interface Reduce<T> {  
  
    T apply(T e1, T e2);  
  
    public static <T> T of(List<T> list, Reduce<T> fun){  
        assert !list.isEmpty();  
        T res= list.get(0);  
        for (int i= 1; i < list.size(); i++){ res = fun.apply(res, list.get(i)); }  
        return res;  
    }  
}  
  
public static void main(String[] arg){  
  
    System.out.println( {  
        List<Integer> list= List.of(1,2,3,4,5,6,7,3);  
        Reduce<T> fun= (a,b)-> a + b;  
        T res= 1;  
        res = fun.apply (res, list.get(1));  
        for (int i= 2; i < 8; i++){ res = fun.apply(res, list.get(i)); }  
        return res;  
    });  
}
```


Guided exercise, Reduce.of

```
public interface Reduce<T> {  
  
    T apply(T e1, T e2);  
  
    public static <T> T of(List<T> list, Reduce<T> fun){  
        assert !list.isEmpty();  
        T res= list.get(0);  
        for (int i= 1; i < list.size(); i++){ res = fun.apply(res, list.get(i)); }  
        return res;  
    }  
}  
  
public static void main(String[] arg){  
  
    System.out.println( {  
        List<Integer> list = List.of(1,2,3,4,5,6,7,3);  
        Reduce<T> fun= (a,b)-> a + b;  
        T res= 1;  
        res = fun.apply (res, 2);  
        for (int i= 2; i < 8; i++){ res = fun.apply(res, list.get(i)); }  
        return res;  
    });  
}
```

Guided exercise, Reduce.of

```
public interface Reduce<T> {  
  
    T apply(T e1, T e2);  
  
    public static <T> T of(List<T> list, Reduce<T> fun){  
        assert !list.isEmpty();  
        T res= list.get(0);  
        for (int i= 1; i < list.size(); i++){ res = fun.apply(res, list.get(i)); }  
        return res;  
    }  
}  
  
public static void main(String[] arg){  
  
    System.out.println( {  
        List<Integer> list= List.of(1,2,3,4,5,6,7,3);  
        Reduce<T> fun= (a,b)-> a + b;  
        T res= 1;  
        res = ((a,b)-> a + b).apply(res, 2);  
        for (int i= 2; i < 8; i++){ res = fun.apply(res, list.get(i)); }  
        return res;  
    });  
}
```

Guided exercise, Reduce.of

```
public interface Reduce<T> {  
  
    T apply(T e1, T e2);  
  
    public static <T> T of(List<T> list, Reduce<T> fun){  
        assert !list.isEmpty();  
        T res= list.get(0);  
        for (int i= 1; i < list.size(); i++){ res = fun.apply(res, list.get(i)); }  
        return res;  
    }  
}  
  
public static void main(String[] arg){  
  
    System.out.println( {  
        List<Integer> list= List.of(1,2,3,4,5,6,7,3);  
        Reduce<T> fun= (a,b)-> a + b;  
        T res= 1;  
        res = res + 2;  
        for (int i= 2; i < 8; i++){ res = fun.apply(res, list.get(i)); }  
        return res;  
    });  
}
```

Guided exercise, Reduce.of

```
public interface Reduce<T> {  
  
    T apply(T e1, T e2);  
  
    public static <T> T of(List<T> list, Reduce<T> fun){  
        assert !list.isEmpty();  
        T res= list.get(0);  
        for (int i= 1; i < list.size(); i++){ res = fun.apply(res, list.get(i)); }  
        return res;  
    }  
}  
  
public static void main(String[] arg){  
  
    System.out.println( {  
        List<Integer> list= List.of(1,2,3,4,5,6,7,3);  
        Reduce<T> fun= (a,b)-> a + b;  
        T res= 3;  
  
        for (int i= 2; i < 8; i++){ res = fun.apply(res, list.get(i)); }  
        return res;  
    });  
}
```

Guided exercise, Reduce.of

```
public interface Reduce<T> {

    T apply(T e1, T e2);

    public static <T> T of(List<T> list, Reduce<T> fun){
        assert !list.isEmpty();
        T res= list.get(0);
        for (int i= 1; i < list.size(); i++){ res = fun.apply(res, list.get(i)); }
        return res;
    }
}

public static void main(String[] arg){

    System.out.println( {
        List<Integer> list= List.of(1,2,3,4,5,6,7,3);
        Reduce<T> fun= (a,b)-> a + b;
        T res= 3;
        res = fun.apply(res, 3);
        for (int i= 3; i < 8; i++){ res = fun.apply(res, list.get(i)); }
        return res;
    });
}
```

Guided exercise, Reduce.of

```
public interface Reduce<T> {  
  
    T apply(T e1, T e2);  
  
    public static <T> T of(List<T> list, Reduce<T> fun){  
        assert !list.isEmpty();  
        T res= list.get(0);  
        for (int i= 1; i < list.size(); i++){ res = fun.apply(res, list.get(i)); }  
        return res;  
    }  
}  
  
public static void main(String[] arg){  
  
    System.out.println( {  
        List<Integer> list= List.of(1,2,3,4,5,6,7,3);  
        Reduce<T> fun= (a,b)-> a + b;  
        T res= 3;  
        res = res + 3;  
        for (int i= 3; i < 8; i++){ res = fun.apply(res, list.get(i)); }  
        return res;  
    });  
}
```

Guided exercise, Reduce.of

```
public interface Reduce<T> {  
  
    T apply(T e1, T e2);  
  
    public static <T> T of(List<T> list, Reduce<T> fun){  
        assert !list.isEmpty();  
        T res= list.get(0);  
        for (int i= 1; i < list.size(); i++){ res = fun.apply(res, list.get(i)); }  
        return res;  
    }  
}  
  
public static void main(String[] arg){  
  
    System.out.println( {  
        List<Integer> list= List.of(1,2,3,4,5,6,7,3);  
        Reduce<T> fun= (a,b)-> a + b;  
        T res= 6;  
  
        for (int i= 3; i < 8; i++){ res = fun.apply(res, list.get(i)); }  
        return res;  
    });  
}
```

Guided exercise, Reduce.of

```
public interface Reduce<T> {  
  
    T apply(T e1, T e2);  
  
    public static <T> T of(List<T> list, Reduce<T> fun){  
        assert !list.isEmpty();  
        T res= list.get(0);  
        for (int i= 1; i < list.size(); i++){ res = fun.apply(res, list.get(i)); }  
        return res;  
    }  
}  
  
public static void main(String[] arg){  
  
    System.out.println( {  
        List<Integer> list = List.of(1,2,3,4,5,6,7,3);  
        Reduce<T> fun = (a,b)->a+b;  
        T res = 6;  
        res = fun.apply(res, 4);  
        for(int i=4;8;i++){ res = fun.apply(res, list.get(i)); }  
        return res;  
    });  
}
```


Guided exercise, Reduce.of

```
public interface Reduce<T> {  
  
    T apply(T e1, T e2);  
  
    public static <T> T of(List<T> list, Reduce<T> fun){  
        assert !list.isEmpty();  
        T res= list.get(0);  
        for (int i= 1; i < list.size(); i++){ res = fun.apply(res, list.get(i)); }  
        return res;  
    }  
}  
  
public static void main(String[] arg){  
  
    System.out.println( {  
        List<Integer> list = List.of(1,2,3,4,5,6,7,3);  
        Reduce<T> fun = (a,b)->a+b;  
        T res = 10;  
        res = fun.apply(res, 5);  
        for(int i=5;8;i++){ res = fun.apply(res, list.get(i)); }  
        return res;  
    });  
}
```

Guided exercise, Reduce.of

```
public interface Reduce<T> {  
  
    T apply(T e1, T e2);  
  
    public static <T> T of(List<T> list, Reduce<T> fun){  
        assert !list.isEmpty();  
        T res= list.get(0);  
        for (int i= 1; i < list.size(); i++){ res = fun.apply(res, list.get(i)); }  
        return res;  
    }  
}  
  
public static void main(String[] arg){  
  
    System.out.println( {  
        List<Integer> list = List.of(1,2,3,4,5,6,7,3);  
        Reduce<T> fun = (a,b)->a+b;  
        T res = 15;  
        res = fun.apply(res, 6);  
        for(int i=6;8;i++){ res = fun.apply(res, list.get(i)); }  
        return res;  
    });  
}
```

Guided exercise, Reduce.of

```
public interface Reduce<T> {  
  
    T apply(T e1, T e2);  
  
    public static <T> T of(List<T> list, Reduce<T> fun){  
        assert !list.isEmpty();  
        T res= list.get(0);  
        for (int i= 1; i < list.size(); i++){ res = fun.apply(res, list.get(i)); }  
        return res;  
    }  
}  
  
public static void main(String[] arg){  
  
    System.out.println( {  
        List<Integer> list = List.of(1,2,3,4,5,6,7,3);  
        Reduce<T> fun = (a,b)->a+b;  
        T res = 21;  
        res = fun.apply(res, 7);  
        for(int i=7;8;i++){ res = fun.apply(res, list.get(i)); }  
        return res;  
    });  
}
```

Guided exercise, Reduce.of

```
public interface Reduce<T> {  
  
    T apply(T e1, T e2);  
  
    public static <T> T of(List<T> list, Reduce<T> fun){  
        assert !list.isEmpty();  
        T res= list.get(0);  
        for (int i= 1; i < list.size(); i++){ res = fun.apply(res, list.get(i)); }  
        return res;  
    }  
}  
  
public static void main(String[] arg){  
  
    System.out.println( {  
        List<Integer> list = List.of(1,2,3,4,5,6,7,3);  
        Reduce<T> fun = (a,b)->a+b;  
        T res = 28;  
        res = fun.apply(res, 3);  
        for(int i=8;8;i++){ res = fun.apply(res, list.get(i)); }  
        return res;  
    });  
}
```

Guided exercise, Reduce.of

```
public interface Reduce<T> {  
  
    T apply(T e1, T e2);  
  
    public static <T> T of(List<T> list, Reduce<T> fun){  
        assert !list.isEmpty();  
        T res= list.get(0);  
        for (int i= 1; i < list.size(); i++){ res = fun.apply(res, list.get(i)); }  
        return res;  
    }  
}  
  
public static void main(String[] arg){  
  
    System.out.println( {  
        List<Integer> list = List.of(1,2,3,4,5,6,7,3);  
        Reduce<T> fun = (a,b)->a+b;  
        T res = 31;  
  
        for(int i=8;8;i++){ res = fun.apply(res, list.get(i)); }  
        return res;  
    });  
}
```

Guided exercise, Reduce.of

```
public interface Reduce<T> {  
  
    T apply(T e1, T e2);  
  
    public static <T> T of(List<T> list, Reduce<T> fun){  
        assert !list.isEmpty();  
        T res= list.get(0);  
        for (int i= 1; i < list.size(); i++){ res = fun.apply(res, list.get(i)); }  
        return res;  
    }  
}  
  
public static void main(String[] arg){  
  
    System.out.println( {  
        List<Integer> list = List.of(1,2,3,4,5,6,7,3);  
        Reduce<T> fun = (a,b)->a+b;  
        T res = 31;  
  
        return res;  
    });  
}
```

Guided exercise, Reduce.of

```
public interface Reduce<T> {  
  
    T apply(T e1, T e2);  
  
    public static <T> T of(List<T> list, Reduce<T> fun){  
        assert !list.isEmpty();  
        T res= list.get(0);  
        for (int i= 1; i < list.size(); i++){ res = fun.apply(res, list.get(i)); }  
        return res;  
    }  
}  
  
public static void main(String[] arg){  
  
    System.out.println( 31 );  
}
```

Next time

- Optionals